

COINCRAFT RESEARCH

Session 2.

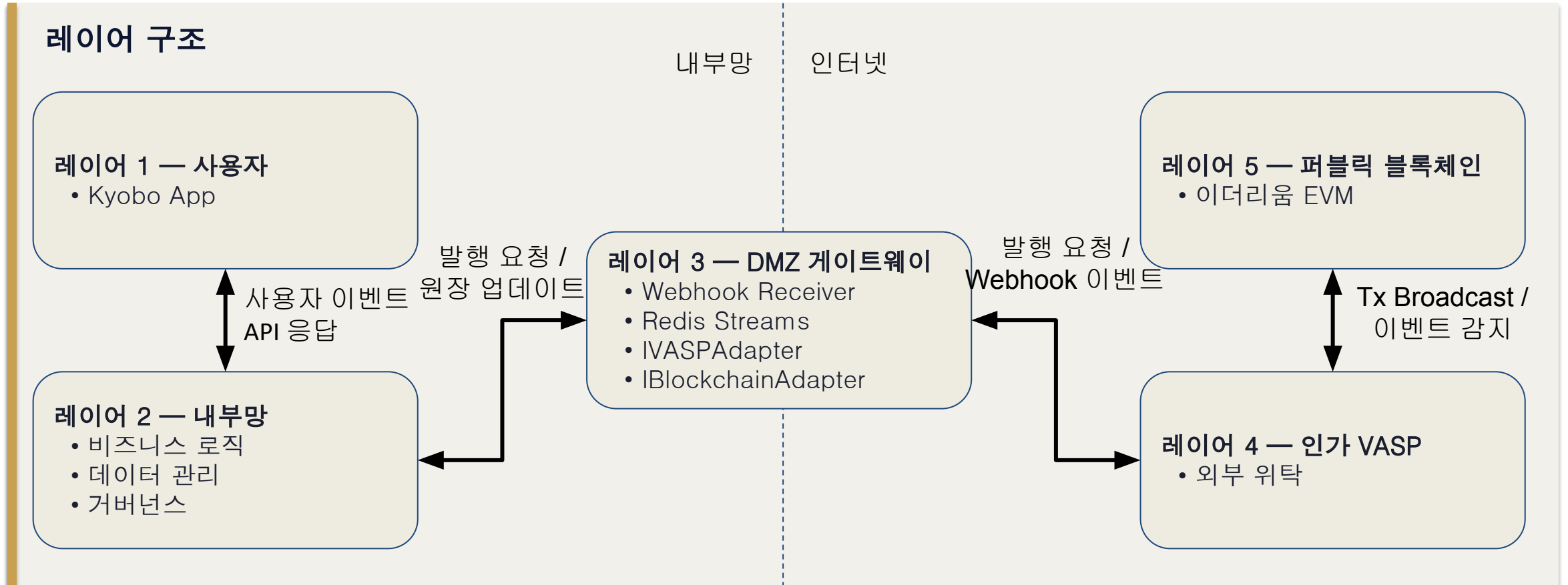
5-Layer 아키텍처 기술 상세

신뢰 경계 · 레이어별 역할 · 핵심 설계 결정

2026.05 | M1 S2 | 1시간

COINCRAFT
ACADEMY

5-Layer 아키텍처 — 신뢰 경계로 시스템을 나눈다



신뢰 경계 = 보안 설계의 기본. 내부망과 외부망 사이에 DMZ를 두는 것은 금융 IT 거버넌스 필수 요건.

5-Layer 아키텍처 — 신뢰 경계로 시스템을 나눈다

신뢰 경계 — 레이어마다 신뢰 수준이 다르다

레이어 1 — 사용자

→ 신뢰 안 함, 모든 입력 검증 필요, JWT 인증 필수

레이어 2 — 내부망

→ 완전 신뢰, 당사 직접 소유·운영, 외부 직접 접근 불가

레이어 3 — DMZ 게이트웨이

→ 부분 신뢰, 외부와 내부 사이 완충, ISMS-P 망 분리 요건 충족

레이어 4 — 인가 VASP

→ 계약 신뢰, SLA 기반 책임, API 인증·서명 검증

레이어 5 — 퍼블릭 블록체인

→ 수학적 신뢰, 코드가 법, TX 서명 = 권한 증명

5-Layer 아키텍처 — User Layer

사용자 레이어

Phase 1-2-3 공통

당사 앱

(예시) 이벤트 달성·혜택 조회 / NFT 보유 현황 확인

 [딥링크 / WalletConnect SDK](#)
TX 서명 요청 시 지갑 팝업 UX

당사 소유

사용자 지갑

월렛원 Web3 연동 검토 or 앱 내장 지갑 (VASP 선정 후 확정)

미확정

VASP 선정 결과에 따라 구조 결정 — 월렛원 선택 시 당사 앱에 Web3 지갑 내장, 위 박스 통합 / MetaMask 딥링크 or 코다 선택 시 당사 앱·사용자 지갑 분리 유지



5-Layer 아키텍처 — Private Network



5-Layer 아키텍처 — DMZ Network

DMZ — 게이트웨이 레이어

Phase 2-3: Consumer 확장만으로 체인 추가

Webhook Receiver

온체인 이벤트 수신
즉시 202 응답 + 큐 적재
(직접 DB 처리 금지)

당사 소유

Redis Streams / Kafka

수신·처리 완전 분리
역등성 보장
Dead Letter Queue
기존 Kafka 인프라 존재 시 교체

미확정

Event Consumer

NFTIssued·NFTBurned 처리
원장 업데이트 + 알림
txHash+logIndex 역등기

당사 소유

VASP 추상화 레이어

VASP API 단일 인터페이스
재시도·Idempotency Key
Phase 3: 직접 Custody로 교체

당사 소유

IBlockchainAdapter

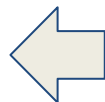
체인 무관 추상화 — 발행·소각·잔액 조회·TX 검증·이벤트 구독
EVM·XRPL·Circle ARC 교체
시 비즈니스 로직 무변경
IVaspAdapter와 동일한 Strategy Pattern

당사 소유

Private RPC Node

전용 EVM 노드
BaaS 위탁 또는 자체구축
(VASP 선정 후 확정)

미확정



5-Layer 아키텍처 — DMZ Network

인가 VASP — 외부 커스터디 (Phase 1·2 위탁)

Phase 3: 당사 직접 Custody 인가로 전환

△ 후보 검토 중 (4월 내 선정 예정) — 월렛원 (IR팀 검토, B2C·신한은행 레퍼런스) · 코다 (당사 투자사, 금융권 Custody) · EQBR (신사업기획팀 검토, 엔터프라이즈 특화)

🔍 VASP 선정 결과에 따른 구조 변경 — 멀티시그·보조키 정책: 월렛원(제어 제한 가능성) · 코다(MPC 기반, 계약 조건 따라 상이) · EQBR(자유도 높음). 당사 보조키 미보유 시 VASP 폐업·해킹 시 자산 접근 불가 리스크

커스터디 지갑

NFT 발행·전송·소각용
핫월렛(20%) / 콜드월렛(80%)
특금법 §10의2 요건

VASP 소유

프라이빗 키 (주키)

MPC or HSM 보관
당사 접근 불가 (Phase 1·2)

VASP 소유

멀티시그 주키

Gnosis Safe 2-of-3
당사 보조키와 쌍

당사+VASP 공유

Node/RPC · BaaS

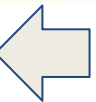
블록체인 인프라 위탁
월렛원·EQBR BaaS 검토
or 당사 자체구축

미확정

TX 서명·브로드캐스트

당사 요청 수신 →
서명 후 메인넷 전송

VASP 실행



5-Layer 아키텍처 — Internet (Blockchain)

퍼블릭 블록체인 — EVM (Ethereum Mainnet)

Phase 2 · 3 (검토 중): Circle Arc | XRPL 추가

ERC-1155 NFT 컨트랙트

발행·소각·벌크 전송

UUPS · AccessControl · Pausable

오너십: 당사

당사 소유

거버넌스 컨트랙트

정책 변경·권한 관리

Gnosis Safe 멀티시그 실행

당사+VASP 공유

Chainlink Oracle

KRW/USD 환율 피드

회로차단기 + Fallback

Phase 2 USDC 결제 필요

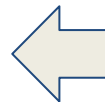
Phase 2 이후 활성화

온체인 이벤트

Transfer · Mint · Burn

Webhook → DMZ로 전달

블록체인



당사 구현 vs 외부 위탁 — 경계를 먼저 정해야 한다

당사 구현 (이 과정의 범위)

비즈니스 로직

- 이벤트 조건 판단 (건기 n보, 건강검진 완료 등)
- NFT 발급 요청 생성
- 사용자-지갑 주소 매핑

데이터 관리

- DMZ 발행 원장 (TX 상태 추적)
- 내부망 영구 보유 원장
- SHA-256 감사 로그

DMZ 게이트웨이

- Webhook Receiver
- Redis Streams + DLQ
- IVASPAAdapter
- IBlockchainAdapter (read-only)

거버넌스

- Gnosis Safe 보조키 서명

외부 위탁 (VASP — 이 과정 범위 밖)

스마트 컨트랙트

- KyoboNFT.sol 배포·운영
- ERC-1155 표준 구현
- 업그레이드 패턴 구현 및 실행

TX 처리

- NFT mint/burn/transfer
- 서명 (MPC/HSM)
- 브로드캐스트

키 관리

- 프라이빗 키 보관
- MPC 분산 서명
- HSM 하드웨어 보안

인프라

- Node/RPC BaaS
- 블록체인 노드 운영
- Webhook 발송
- Web3 지갑 SDK

이 경계가 흔들리면 안 된다. Tx키 보관하려면 VASP 인가 취득 필요 (특금법). Phase 3에서 검토.

이벤트 흐름

전체 흐름

① 사용자 앱 → ② 내부망 비즈니스 로직 → ③ IVASPAdapter → ④ VASP API →
⑤ EVM 컨트랙트 이벤트 → ⑥ Webhook 수신 → ⑦ 내부 원장 반영

Write Path (발급 요청)

① 사용자 앱: 건기 10,000보 달성 → 내부망 API 전송

② EventConditionService:
조건 판단 → NFT 발급 결정 → requestId 생성 (UUID) → mint_requests DB 기록 →
REQUESTED

③ IVASPAdapter.mintNFT():
→ ExternalVASPAdapter → VASP REST API 호출 → Idempotency-Key 헤더 첨부 →
SUBMITTED

④ VASP: TX 서명·브로드캐스트 → txHash 반환 → **PENDING**

이벤트 흐름

전체 흐름

① 사용자 앱 → ② 내부망 비즈니스 로직 → ③ IVASPAAdapter → ④ VASP API →
⑤ EVM 컨트랙트 이벤트 → ⑥ Webhook 수신 → ⑦ 내부 원장 반영

온체인 처리

⑤ 이더리움 EVM:

mempool 대기 → PENDING

블록 채굴 → MINED

32블록 추가 → CONFIRMED

KyoboNFT.mintBatch() 실행

TransferSingle 이벤트 발생

→ VASP 이벤트 감지

→ Webhook HTTP POST 발송

이벤트 흐름

전체 흐름

① 사용자 앱 → ② 내부망 비즈니스 로직 → ③ IVASPAAdapter → ④ VASP API →
⑤ EVM 컨트랙트 이벤트 → ⑥ Webhook 수신 → ⑦ 내부 원장 반영

Read Path (이벤트 수신)

⑥ DMZ Webhook Receiver:

- HTTP 202 즉시 반환
- Redis XADD

Event Consumer:

- XREADGROUP으로 소비
- NFTIssuedHandler 실행
- 내부 원장 CONFIRMED
- XACK

사용자 앱:

- 잔액 조회 → 쿠폰 표시
- 사용 가능

EventConditionService — '언제, 누구에게, 어떤 NFT를' 결정

비즈니스 로직이 온체인이 아닌 이유

온체인에 조건 판단 로직 넣으면:

```
if (walkingSteps >= 10000) {  
  mintNFT(user, WALKING_TOKEN);  
}
```

문제:

- ① 조건 변경 시 컨트랙트 업그레이드 → 가스비 + 검증 + 시간
- ② 걸음 수 데이터를 온체인으로 → Oracle 필요 + 비용
- ③ 개인 건강 데이터 온체인 기록 → GDPR·개인정보보호법 위반
- ④ 유연한 비즈니스 규칙 변경 불가 → '2만보 달성 시 2배 쿠폰' 이벤트 → 배포 없이 변경 불가

EventConditionService — '언제, 누구에게, 어떤 NFT를' 결정

오프체인 조건 판단 구조

```
interface ActivityEvent {  
  userId: string;  
  activityType:  
    'WALKING' | 'HEALTH_CHECK' | 'SIGN_UP';  
  value: number; // 걸음 수  
  timestamp: Date;  
}
```

```
// EventConditionService.ts  
if (event.activityType === 'WALKING'  
    && event.value >= 10000) {  
  await issuerService.issueSingle(  
    event.userId,  
    WALKING_TOKEN_ID  
  );  
}
```

- 조건 변경 = 서버 코드 수정 후 배포
- 배포 비용: 없음
- 개인 데이터: 내부 DB에만 보관
- 온체인: 발행 결과만 기록

BulkIssueService — 10만 건 동시 발행을 어떻게 처리하는가

문제: 한 번에 10만 건 보내면

VASP API에 10만 건 요청 동시 전송

- VASP Rate Limit 초과 (429)
- 또는 VASP 처리 대기열 폭주
- 또는 ERC-1155 mintBatch 블록 가스 한도 초과

$500\text{건} \times \sim 50,000 \text{ gas} = 25,000,000 \text{ gas}$

블록 가스 한도 $\approx 30,000,000 \text{ gas}$

→ 500건이 블록 내 안전 상한선

BulkIssueService 해결

전체 10만 건

↓ 500건 단위 분할

배치 1 (500건) → VASP → 성공 → 원장 업데이트

배치 2 (500건) → VASP → 실패 → 재시도 큐

배치 3 (500건) → VASP → 성공 → ...

- 진행률 추적: 배치 단위로 상태 기록
- 실패 격리: 실패한 배치만 재시도
- 중단 복구: 서버 재시작 시 미완료 배치부터 재개

이벤트 시나리오: 연말 이벤트 당첨자 전체 발행

당첨자 100,000명 → 500건 배치 200개 → 순차 또는 병렬 처리

각 배치: requestId 고유 부여 → Idempotency 보장 → 실패 시 해당 배치만 재시도

전체 완료 시간: 200 배치 $\times$ 15초 = 최대 50분 (병렬 처리로 단축 가능)

DMZ — Node.js를 선택한 이유

DMZ (Node.js)

dmz/ 폴더

위치: ISMS-P DMZ 구간
외부와 내부 사이 완충

담당:

- IVASPAAdapter (VASP 연동)
- Webhook Receiver
- Redis Streams Consumer
- EventConditionService
- BulkIssueService
- DMZ 발행 원장 (mint_requests)
- SHA-256 TX 감사 로그

Node.js 선택 이유

- ① 블록체인 도구와 같은 스택
ethers.js · Hardhat · viem
→ M6~M8 컨트랙트 코드와 타입·ABI 공유 가능
- ② TypeScript 타입 안전성
컴파일 타임에 VASP API 응답 구조 오류 사전 차단
- ③ JSON I/O 중심 작업에 적합
VASP API · Webhook · Redis 모두 JSON 기반
- ④ 빠른 개발·배포 주기
npm 패키지 생태계 풍부

Node.js를 선택한 핵심 이유는 블록체인 개발 도구(ethers.js·Hardhat)와 같은 언어 스택을 유지하기 위함

내부망 — Java Spring Boot를 선택한 이유

내부망 (Java Spring Boot) internal/ 폴더

위치: 교보생명 내부망
외부 직접 접근 불가

담당:

- 영구 NFT 보유 원장
- 금융 감사 로그 (5년 보관)
- Row Level Security
- 사용자 인증·식별
- Admin 대시보드
- 레거시 시스템 연동

Java Spring Boot 선택 이유

- ① 기존 스택 그대로 활용
내부 시스템이 이미 Java 기반
- ② 레거시 연동 용이
보험 코어뱅킹·퇴직연금 시스템과 직접 연동
- ③ 검증된 금융 보안
Spring Security
Row Level Security
- ④ 트랜잭션·감사 성숙도
5년 보관 감사 로그 / 금융권 감사 도구 호환

역할이 달라서 기술도 다르다. DMZ = 블록체인 생태계 통합 / 내부망 = 기존 금융 인프라 연동.

DMZ 발행 원장 — mint_requests 테이블과 TX 상태 추적

mint_requests 테이블 구조

```
CREATE TABLE mint_requests (  
  id          UUID PRIMARY KEY,  
  request_id  VARCHAR UNIQUE, -- Idempotency key  
  user_id     VARCHAR,  
  token_id    BIGINT,  
  amount      BIGINT,  
  wallet_addr VARCHAR,  
  status      VARCHAR,  
  -- REQUESTED·SUBMITTED·PENDING·MINED·CONFIRMED·FINALIZED·FAILED·REORGED  
  tx_hash     VARCHAR,  
  block_number BIGINT,  
  gas_used     BIGINT,  
  error_msg    TEXT,  
  retry_count  INT DEFAULT 0,  
  created_at   TIMESTAMPTZ,  
  updated_at   TIMESTAMPTZ  
);
```

DMZ 발행 원장 — mint_requests 테이블과 TX 상태 추적

상태 전이 규칙

REQUESTED → SUBMITTED

조건: VASP API 호출 성공 + txHash 수신

액션: tx_hash, updated_at 기록

SUBMITTED → PENDING

조건: VASP가 mempool 진입 확인

PENDING → MINED

조건: Webhook NFTIssued 수신

액션: block_number, gas_used 기록

MINED → CONFIRMED

조건: $\text{block_number} + 32 < \text{currentBlock}$

액션: 내부 원장 CONFIRMED 업데이트

PENDING → FAILED

조건: 5분 타임아웃 + 가스범프 실패

액션: DLQ 이관 + 알림

* → REORGED

조건: REORG Webhook 수신

액션: 원장 롤백 + 재발행 검토

영구 보유 원장 — 온체인 상태의 내부 미러

왜 내부 원장이 필요한가

온체인 직접 조회:

- eth_call(balanceOf, userAddr, tokenId)
- 매 조회마다 RPC 호출
- 사용자 100만 명 = 100만 RPC
- 비용 + 레이턴시 (200~500ms)
- 교보 앱 응답 속도 저하

내부 원장 방식:

- DB 조회 → 수 밀리초
- 온체인과 주기적 Reconcile
- 차이 발생 시 알림 + 수동 검토

nft_holdings 테이블 (내부망)

nft_holdings:

```
user_id  VARCHAR
token_id BIGINT
balance  BIGINT    -- 현재 보유 수량
last_sync TIMESTAMP -- 마지막 온체인 동기화
PRIMARY KEY (user_id, token_id)
```

- balanceOf(user, tokenId) 대신
SELECT balance FROM nft_holdings
WHERE user_id = ? AND token_id = ?

Reconcile 주기

일반: 매 블록마다 Webhook으로 동기화 (실시간)

정기 검증: 1일 1회 전체 잔액 온체인 대조 → 불일치 감지 시 알림

감사 시: 특정 시점 blockNumber 지정 → eth_call(balanceOf, addr, id, {blockTag}) 재현

감사 로그 — DMZ + 내부망 이중 구조

DMZ 감사 로그 (TX 이벤트 감사)

파일: AuditLogService.ts

기록 대상:

- mint_requests 상태 전이
- VASP API 호출·응답
- Webhook 수신
- Consumer 처리 결과
- 가스범프 이벤트
- DLQ 이관

구조:

id, event_type, payload,
tx_hash, status_from, status_to,
prev_hash, hash (SHA-256)

특성:

- append-only (UPDATE/DELETE 없음)
- SHA-256 체인으로 변조 감지

감사 로그 — DMZ + 내부망 이중 구조

내부망 금융 감사 로그 (규제 대응)

파일: AuditLogService.java

기록 대상:

- NFT 보유 원장 모든 변경
- 사용자 인증 이벤트
- Admin 조작 이력
- Reconcile 결과
- 이상 감지 알림

구조:

동일 SHA-256 체인 + Row Level Security
(감사팀만 읽기 가능)

내부망 금융 감사 로그 (규제 대응)

보관 요건:

- 전자금융거래법: 5년
- ISMS-P: 접근 로그 포함
- 내부통제: 변경 이력 전수

특성:

- PostgreSQL append-only 설정
- 삭제 권한 없음 (DBA도 불가)

두 감사 로그는 서로 다른 규제 목적. DMZ = TX 처리 추적 / 내부망 = 금융 규제 준수 기록.

거버넌스 — 당사 보조키의 역할과 사용 시나리오

보조키가 필요한 작업들

- ① 대량 NFT 발행 (10만 건 이상)
 - 임계값 초과 시 당사 서명 필요
- ② KyoboNFT 컨트랙트 업그레이드
 - `upgradeToAndCall()` 실행
 - `UPGRADER_ROLE = Gnosis Safe`
- ③ 긴급 정지
 - `pause()` 실행
 - `PAUSER_ROLE` 서명
- ④ 비정상 TX 강제 취소
 - 고가스 덮어쓰기

실제 업그레이드 시나리오

- 1. VASP(월렛원)가 SafeTx 제안
 - to: `KyoboNFTProxy`
 - data: `upgradeToAndCall(newImpl)`
- 2. 당사 Admin 대시보드 알림
- 3. 당사 보안팀: EIP-712 구조 확인
 - '어떤 컨트랙트를 어떻게 업그레이드'
 - 내용 확인 후 K1 서명
- 4. 2-of-3 충족 (K1 교보 + K2 월렛원)
 - `Gnosis Safe execTransaction` 실행

Admin 대시보드 — 당사가 직접 구현하는 이유

VASP가 제공하는 대시보드에만 의존하면: VASP 장애 시 모니터링 불가 / 내부통제 증거 없음
당사 Admin 대시보드: 발급 현황·FAILED TX·Gnosis Safe 서명 요청·Reconcile 결과 — 독립적 모니터링

DMZ — ISMS-P가 요구하는 망 분리의 실체

DMZ 없이 직접 연결하면

내부망 서버 $\longleftrightarrow$ 월렛원 API
내부망 서버 $\longleftrightarrow$ 블록체인 노드

문제:

- ISMS-P 망 분리 요건 위반
- 외부 공격 시 내부망 직접 노출
- VASP 장애 = 내부망 전체 영향
- 금감원 심사 통과 불가

DMZ 완충 지대

내부망 $\longleftrightarrow$ [DMZ] $\longleftrightarrow$ 월렛원 API
[DMZ] $\longleftrightarrow$ 블록체인 노드

- DMZ가 외부와 내부 사이 중재
- 외부 공격: DMZ까지만 노출
- VASP 장애: DMZ가 흡수
- ISMS-P 망 분리 요건 충족
- 금감원 심사 통과 가능

ISMS-P DMZ 요건

정보보호관리체계(ISMS-P) 인증: 외부 인터넷 서비스와 내부망 사이에 DMZ 구성 의무
방화벽 규칙: DMZ $\rightarrow$ 내부망 인바운드 최소화 / 내부망 $\rightarrow$ DMZ 아웃바운드만 허용
접근 로그: DMZ 통과 모든 트래픽 기록 $\rightarrow$ 감사 시 제출

DMZ는 선택이 아니다. 금융 IT 거버넌스에서 요구하는 보안 아키텍처 필수 요소.

Webhook Receiver — 202 패턴으로 이벤트를 안전하게 수신

202 없이 동기 처리하면

VASP → POST /webhook/nft-issued

↓ 핸들러가 DB 직접 업데이트

↓ (처리 시간 500ms~3초)

VASP: 응답 지연 → 타임아웃 (보통 3~5초)

→ 재전송 (retry)

→ 같은 이벤트 2번 수신

→ 원장 2번 업데이트

→ 잔액 2배 기록

이것이 '중복 처리' 문제

202 패턴

VASP → POST /webhook/nft-issued

↓ 즉시 HTTP 202 Accepted 반환

body: { received: true }

↓ (응답 완료, VASP 안심)

↓ 비동기: Redis XADD

blockchain-events '* event {...}'

VASP: 202 수신 → 재전송 없음

Consumer: Redis에서 꺼내서 처리

→ 처리 실패해도 Stream에 남아있음

→ 재처리 가능

Webhook 인증 — VASP 서명 검증 필수

VASP는 Webhook 헤더에 HMAC-SHA256 서명 첨부 → Receiver가 공유 시크릿으로 검증

검증 실패 시: 즉시 401 반환 (Redis에 적재 안 함) → 악의적 Webhook 차단

Redis Streams — 일반 Pub/Sub이나 MQ와 무엇이 다른가

항목	일반 Pub/Sub · MQ	Redis Streams
소비 방식	꺼내면 메시지 사라짐	소비 후에도 스트림에 남음
Consumer Group	없음 (단일 소비자)	있음 (여러 Consumer 분산 처리)
장애 복구	메시지 유실	PEL에서 미ACK 메시지 복구
재처리	불가 (이미 사라짐)	가능 (ID로 특정 메시지 재처리)
순서 보장	없음	ID(timestamp-seq) 기반 순서 보장
메시지 ID	없음	1620000000000-0 형식 (시간 내장)
운영 비용	낮음	낮음 (Redis 이미 사용 중)
처리량	높음	높음 (Redis 인메모리)

Kafka 사용, 교보 Phase 1 처리량(수천 건/시간)에 Redis Streams로 충분. 운영 비용도 낮다.

Consumer Group — At-least-once 처리 보장 메커니즘

PEL (Pending Entry List)

XREADGROUP로 메시지 읽을 때:

- PEL에 자동 등록 (메시지 ID + 소비자 이름 + 시각)

XACK로 처리 완료 알릴 때:

- PEL에서 해당 메시지 제거

XAUTOCLAIM 실행 시:

- PEL에서 min-idle-time 초과

- 미ACK 메시지 발견

- 다른 Consumer로 재할당

- Consumer 장애로 ACK 못 해도 다른 Consumer가 재처리

- 메시지 유실 없음

Consumer Group — At-least-once 처리 보장 메커니즘

멱등 키 — 중복 처리 방지

Consumer가 같은 메시지를 두 번 처리할 수 있다 (At-least-once의 의미)

중복이 생기는 경우:

- Consumer 장애 후 재할당
- VASP가 같은 이벤트 재전송
- 네트워크 재시도

해결 — 멱등 키: txHash + logIndex

처리 전 DB 조회:

```
SELECT id FROM processed_events  
WHERE idempotency_key = ?
```

이미 있으면 → skip + ACK

없으면 → 처리 + INSERT + ACK

→ 몇 번 중복 수신해도 원장은 1번만 업데이트

DLQ — 실패 이벤트를 버리지 않는다

DLQ 없으면

처리 실패 이벤트

- 재시도 3회
- 모두 실패
- 메시지 드롭 (유실)

결과:

- 고객 쿠폰 미발행
- 내부 원장 불일치
- 감사 로그 누락
- 어떤 이벤트가 유실됐는지 모름
- 금융 규제 위반

DLQ 전략

재시도 3회 실패

- XADD blockchain-events-dlq
(원본 이벤트 전체 + 실패 이유 + 스택트레이스)
- 운영팀 알림 (Slack·이메일)

원인 수정 후:

- 수동 DLQ 재처리 명령
- 해당 이벤트 재투입

자동 재처리 금지 이유:

- 같은 버그 → 무한 루프
- 시스템 자원 낭비

DLQ 모니터링

Admin 대시보드: DLQ 건수 실시간 표시 → 0이 정상 / 1 이상이면 알림

정기 점검: 일 1회 DLQ 스캔 → 원인 파악 → 수정 배포 → 재처리

SLA: DLQ 이벤트 48시간 내 처리 완료 목표

두 가지 추상화 — IVASPAdapter와 IBlockchainAdapter는 다르다

IVASPAdapter 규제 레이어 추상화

Phase 1: ExternalVASPAdapter

- VASP REST API 호출
- Idempotency-Key 헤더
- 응답 → 내부 원장 업데이트

Phase 3: KyoboVASPAdapter (가정)

- 교보 직접 VASP 인가 취득 후
- 자체 키 관리·TX 서명
- IssuerService 코드 변경 없음

교체 시점: 비즈니스·법적 결정
VASP 인가 취득 여부에 따라

핵심 메서드:

mintNFT(), burnNFT()
getWallet(), getTxStatus()

IBlockchainAdapter 기술 레이어 추상화

Phase 1: EVMAdapter

- 이더리움 EVM
- ethers.js 기반
- gasUsed 존재
- Finality ~12분

미래: XRPLAdapter

- XRPL 네트워크
- xrpl.js 기반
- gasUsed 없음
- Finality ~4초

교체 시점: 기술적 결정
어떤 블록체인으로 갈지에 따라

핵심 메서드:

queryEvents(), getReceipt()
subscribeEvents(), getBalance()

이 두 결정은 독립적이다. VASP는 유지하면서 체인만 바꿀 수 있다. 체인은 유지하면서 VASP만 바꿀 수 있다.

IBlockchainAdapter — Phase 1에서는 read-only만 사용

전체 인터페이스

```
interface IBlockchainAdapter {  
  chainId: string;  
  chainType: 'EVM' | 'XRPL' | 'UTXO' | 'BFT';  
  
  // 연결 상태  
  isConnected(): Promise<boolean>;  
  getBlockNumber(): Promise<number>;  
  
  // NFT 조작 (write)  
  mintNFT(p: MintParams)  
    : Promise<TransactionReceipt>;  
  mintNFTBatch(p: MintBatchParams)  
    : Promise<TransactionReceipt>;  
  burnNFT(p: BurnParams)  
    : Promise<TransactionReceipt>;  
  
  // 조회 (read)  
  getBalance(addr, op, tokenId)  
    : Promise<bigint>;  
  getReceipt(txHash)  
    : Promise<TransactionReceipt | null>;  
  
  // 이벤트  
  subscribeEvents(...): Promise<()=>void>;  
  queryEvents(...): Promise<ChainEvent[ ]>;  
}
```


IBlockchainAdapter — Phase 1에서는 read-only만 사용

Phase 1: read-only만 사용하는 이유

write 메서드 (mintNFT 등):

- 인터페이스에 존재
- Phase 1에서는 미호출
- 월렛원이 TX 서명·발송

⚠ Phase 1에서 mintNFT() 직접 호출 시:

- 개인키가 당사 서버에 있어야 함
- 특금법 VASP 인가 필요
- 인가 없이 호출 = 법률 위반

Phase 1에서 실제 사용:

- queryEvents():
missed event 복구 (Webhook 누락 시)
- getReceipt():
txHash로 REVERT 감지
- subscribeEvents():
WebSocket 보조 구독
- getBalance():
Reconcile 시 온체인 대조

Phase 3 이후:

- 교보 VASP 인가 취득 시
- mintNFT() 직접 호출 활성화

Private RPC Node — 퍼블릭 RPC로 부족한 이유

퍼블릭 RPC (Alchemy 무료) 문제

Rate Limit:

초당 300 요청 제한

→ Reconcile 100만 건 조회 시 제한 초과

데이터 의존성:

Alchemy 장애 = 우리 서비스 장애

SLA 없음 (무료 플랜)

프라이버시:

어떤 주소를 언제 조회하는지

Alchemy가 알게 됨

→ 고객 데이터 간접 노출

Phase 1 구성

옵션 A: 월렛원 BaaS 사용

→ 월렛원이 제공하는 전용 RPC

→ SLA 보장

→ 별도 인프라 불필요

옵션 B: 자체 노드 운영

→ Geth/Nethermind 직접 실행

→ 완전한 데이터 독립성

→ 높은 운영 비용

(서버 4코어 32GB RAM + 8TB SSD)

Phase 1: 옵션 A 선택

→ 운영 비용 절감

→ VASP SLA에 포함

인가 VASP — 특금법이 만든 제약

특금법 (특정금융정보법)

가상자산사업자(VASP) 정의:

- 가상자산 매매·교환
- 가상자산 이전 (전송·보관)
- 가상자산 보관·관리
- 가상자산 매매·교환 중개

→ NFT 발행·보관을 직접 하려면
금융정보분석원(FIU) 신고 필요
(사실상 인가)

신고 요건:

- 정보보호관리체계(ISMS) 인증
- 실명계좌 확인서
- AML(자금세탁방지) 시스템
- 대표자 신원 조회

소요 기간: 6~12개월

소요 비용: 수억 원

인가 VASP — 특금법이 만든 제약

교보생명의 선택

Phase 1: 외부 VASP (월렛원) 위탁

- 월렛원이 VASP 인가 보유
- 교보는 API로만 연동
- 교보는 VASP 인가 불필요

이 결정의 의미:

- 개인키를 교보가 보관 안 함
- TX 서명을 교보가 안 함
- 법적 책임은 월렛원
- 교보는 비즈니스 로직만 담당

월렛원이 담당:

- VASP 인가 유지
- AML 의심 TX 모니터링
- Travel Rule 준수
- 고객 지갑 생성·관리

Phase 3 검토:

- 교보 직접 VASP 인가 취득
- KyoboVASPAdapter 전환

커스터디 지갑 — 핫월렛과 콜드월렛 분리

핫월렛 (Hot Wallet) — 20%

특성:

- 온라인 상태 상시 유지
- 서명 즉시 가능
- 일상적인 발행·소각에 사용

위험:

- 온라인 = 해킹 노출 가능성
- 20% 상한 = 피해 제한

월렛원 구성:

- HSM 연결된 서버
- MPC 키 파편 일부 온라인
- 자동 서명 가능

콜드월렛 (Cold Wallet) — 80%

특성:

- 오프라인 또는 에어갭
- 대부분 자산 보관
- 서명 시 수동 개입 필요

사용 시점:

- 대규모 자산 이동
- 비상 복구
- 주기적 핫월렛 보충

월렛원 구성:

- HSM 오프라인 보관
- 물리적 금고
- 다중 지역 분산

80-20 규칙의 의미

핫월렛 해킹 최악의 시나리오: 전체 자산의 20%만 위험

콜드월렛 접근: 물리적 보안 + 다인 승인 필요 → 해킹 사실상 불가

교보 입장: 이 구조가 SLA에 명시됨 → 월렛원 계약서에 80-20 요건 포함

MPC — 프라이빗 키가 어디에도 존재하지 않는 서명

기존 키 관리의 문제

전통적 방법:

프라이빗 키 = 단일 파일
하나의 장치(HSM)에 보관

위험:

그 장치가 침해되면
→ 전체 키 노출
→ 전체 자산 위험

개선 시도 — 멀티시그:

여러 키로 m-of-n 서명
→ 각 키가 각 장치에
→ 장치 수만큼 키 파일 존재
→ 각 장치 침해 시 해당 키 노출

→ '키가 어딘가에 존재한다'
는 문제 해결 안 됨

MPC (Multi-Party Computation)

키를 수학적으로 N개 파편으로 분할

파편 1 → 장치 A

파편 2 → 장치 B

파편 3 → 장치 C

서명 요청 시:

각 장치가 자신의 파편으로
부분 계산 수행
→ 중간값만 교환 (키 파편 비공개)
→ 완성된 서명만 출력

전체 키는 어디에도 존재 안 함:

장치 A 해킹 → 파편 1만 노출
→ 서명 불가 (m-of-n 미충족)
장치 A + B 해킹 → 2/3 파편
→ 서명 불가 (3-of-3이면)

MPC = 프라이빗 키를 생성하지 않고 서명. 가장 안전한 키 관리 방식.

KyoboNFT.sol — 4개 상속의 이유

ERC1155 Upgradeable

다중 토큰 표준

- 하나의 컨트랙트로 모든 쿠폰 종류 관리
- balanceOf(addr, id)
- mintBatch 지원
- 1 TX = 100명 발행

→ 쿠폰 종류마다
컨트랙트 배포
→ 없애줌

AccessControl Upgradeable

역할 기반 권한

- MINTER_ROLE
→ VASP 계정
- PAUSER_ROLE
→ 보안팀
- UPGRADER_ROLE
→ Gnosis Safe

onlyOwner 단일 키
→ 없애줌

역할 탈취 시
피해 범위 격리

Pausable Upgradeable

긴급 정지

- pause() 호출 시 모든 mint/burn /transfer 차단
- 보안 사고 즉시 대응
- unpause는 별도 프로세스

→ '멈출 수 없는 시스템'의 위험 제거

UUPS Upgradeable

업그레이드 가능

- 버그 수정
- 기능 추가
- 데이터 유지

constructor 없음
→ initialize() 사용

업그레이드 로직
= Impl 안에
→ Proxy 단순

UPGRADER_ROLE
= Gnosis Safe
→ 단독 불가

이 4개 상속은 '금융 시스템이 컨트랙트에 요구하는 최소 요건'이다. 하나라도 빠지면 운영 불가.

온체인 이벤트 — 컨트랙트에서 DMZ까지

ERC-1155 표준 이벤트

```
// Solidity (월렛원 운영)
event TransferSingle(
    address indexed operator,
    address indexed from,
    address indexed to,
    uint256 id,      // tokenId
    uint256 value    // amount
);
```

```
event TransferBatch(
    address indexed operator,
    address indexed from,
    address indexed to,
    uint256[] ids,
    uint256[] values
);
```

from = address(0): mint 이벤트
to = address(0): burn 이벤트
그 외: transfer 이벤트

→ 이 이벤트들이 Webhook으로 전달

온체인 이벤트 — 컨트랙트에서 DMZ까지

이벤트 구독 흐름

온체인 이벤트 발생
(TransferSingle emit)



VASP 노드가 이벤트 감지



VASP → POST /webhook/nft-issued
body: {
 event: 'NFTIssued',
 txHash: '0xabc...',
 logIndex: 0,
 contractAddr: '0xKyobo',
 to: '0xUser',
 tokenId: '...',
 amount: '1'
}



DMZ Webhook Receiver

→ HTTP 202 반환

→ Redis XADD



Consumer Group Worker

→ NFTIssuedHandler

→ mint_requests CONFIRMED

→ nft_holdings +1

→ 감사 로그 기록

→ XACK

ADR 001 — 체인 추상화 레이어 도입 결정

ADR이란

Architecture Decision Record

아키텍처 결정을 문서화한 것:

- 무엇을 결정했는가
- 왜 이 결정을 했는가
- 기각된 대안은 무엇인가
- 이 결정의 영향은 무엇인가

왜 필요한가:

6개월 뒤 '왜 이렇게 했지?'
→ ADR 보면 답이 있음

새 팀원 온보딩:

→ ADR 읽으면 맥락 파악

결정 반복 검토:

→ 원래 기각 이유 확인 후 판단

docs/adr/ 폴더에 저장

ADR 001: IBlockchainAdapter 도입

결정:

IBlockchainAdapter 인터페이스 도입
EVMAdapter, XRPLAdapter 구현체 분리

기각된 대안:

ethers.js를 비즈니스 로직에서 직접 호출

기각 이유:

EVM 종속 코드가 모든 레이어에 산재
→ XRPL 추가 시 비즈니스 로직 전체 수정
→ 단위 테스트 시 실제 노드 필요
→ 체인 교체 비용이 너무 큼

결정의 영향:

+ 체인 추가 = 어댑터 구현만
+ MockAdapter로 테스트 가능
- 인터페이스 설계 초기 비용
- 체인별 특수 기능 표현 제한

ADR 002 — VASP 외부 우선 결정

ADR 002: VASP 외부 위탁

결정:

Phase 1은 외부 인가 VASP (월렛원)
교보 직접 VASP 인가 취득 안 함

기각된 대안:

즉시 교보 자체 VASP 인가 취득

기각 이유:

인가 취득 소요 기간: 6~12개월
→ Phase 1 서비스 출시가 늦어짐
→ 고객 서비스 지연
→ 기회비용

인가 취득 비용:

→ 수억 원 규모
→ Phase 1 ROI 검증 전 투자 리스크

결정의 영향:

- + Phase 1 빠른 출시
- + 초기 비용 절감
- 월렛원 의존성 (단일 장애점)
- VASP 수수료 지속 발생
- Phase 3 전환 시 마이그레이션 필요

ADR 002 — VASP 외부 우선 결정

IVASPAdapter가 이 결정을 지탱하는 이유

만약 IVASPAdapter 없이
월렛원 API를 직접 호출하면:

```
await walletone.mintNFT({  
  apiKey: WALLESTONE_KEY,  
  contractAddr: WALLESTONE_CONTRACT,  
  ...  
})
```

Phase 3 전환 시:

비즈니스 로직 전체에서
walletone → kyoboVASP 교체
→ 수백 개 파일 수정
→ 테스트 전체 재작성

IVASPAdapter 있으면:

```
new IssuerService(  
  new KyoboVASPAdapter() // 교체  
)  
→ IssuerService 코드 변경 없음
```

→ ADR 002의 결정이
IVASPAdapter 설계를 강제함

신뢰 경계 — 레이어 간 통신 규칙

레이어 간 통신 규칙

사용자 앱 → 내부망:

- HTTPS + JWT 인증
- 입력 전수 검증
- 사용자 권한 확인

내부망 → DMZ:

- 내부 API (HTTP)
- 서비스 간 인증 (API Key)
- 외부에서 직접 접근 불가

DMZ → VASP:

- HTTPS
- API Key + HMAC 서명
- Idempotency-Key 헤더
- Rate Limit 준수

VASP → DMZ (Webhook):

- HMAC-SHA256 서명 검증
- IP 화이트리스트
- 202 즉시 반환

각 레이어의 접근 제어

사용자 앱 (신뢰 안 함):

JWT 만료 15분
Refresh Token 7일
Rate Limit: 100 req/min/user

내부망 (완전 신뢰):

방화벽: DMZ에서만 인바운드
DB: 내부망 서버에서만 접근
Admin: VPN + MFA 필수

DMZ (부분 신뢰):

월렛원 IP 화이트리스트
Webhook URL 비공개
Redis: 내부망에서만 접근

퍼블릭 블록체인 (수학적 신뢰):

TX 서명 = 권한 증명
검증 = 노드 전체가 합의
롤백 불가 (FINALIZED 후)

장애 격리 — 한 레이어의 장애가 전체로 퍼지지 않아야 한다

VASP API 장애 시

발생: VASP API 5xx 응답 또는 타임아웃

DMZ 동작:

- 재시도 (지수 백오프)
- 3회 실패 → FAILED
- DLQ 이관
- 운영팀 알림

영향 범위:

- 신규 NFT 발행 불가
- 기존 보유 원장: 정상
- 사용자 쿠폰 조회: 정상
- 내부망: 영향 없음

복구 후:

- DLQ 재처리
- 미발행 건 순차 처리

Redis 장애 시

발생: Redis 다운

DMZ 동작:

- Webhook 수신 후 Redis XADD 실패
- 202 응답은 이미 완료
- 이벤트 유실 위험

대응:

- Redis HA 구성 (Sentinel 또는 Cluster)
- Webhook 재전송
VASP가 72시간 내재전송
- queryEvents로 missed event 복구

복구 후

- 누락 이벤트 수동 재처리

장애 격리 — 한 레이어의 장애가 전체로 퍼지지 않아야 한다

블록체인 REORG 발생 시

발생:

블록 REORG → 처리한 이벤트 무효화

DMZ 동작:

- REORG Webhook 수신
- 해당 txHash 이벤트 역연산
- 내부 원장 REORGED 상태로 업데이트
- 감사 로그에 REORG 사유 기록

재발행 검토:

- REORGED TX 재 확인
- 필요 시 재발행
- requestId 재사용 (Idempotency 보장)

장애 격리 = 어느 한 레이어가 죽어도 다른 레이어는 계속 동작. 이것이 5레이어 분리의 실질적 가치.

Phase 1 → Phase 3 — 이 아키텍처가 어떻게 진화하는가

Phase 1 (현재) 외부 VASP 위탁

기술:

- ExternalVASPAdapter → 월렛원 API
- EVMAdapter (read-only)
- 이더리움 EVM

키 관리:

- VASP MPC/HSM
- KYOBO: 보조키만

컨트랙트:

- VASP 배포·운영

비용 구조:

- VASP 수수료
- 개발 비용 낮음

목표:

서비스 출시
Product-Market Fit 검증

Phase 2 (계획) 멀티체인

확장 기술:

- XRPLAdapter 추가
- CircleAdapter 추가

컨트랙트:

- XRPL XLS-20
- Circle ARC

비용 구조:

- 체인별 VASP 수수료

목표:

- Finality 4초 (XRPL)
- 수수료 없는 발행
- 글로벌 사용자 대응

코드 변화:

- IssuerService: 변경 없음
- 어댑터 구현만 추가

Phase 1 → Phase 3 — 이 아키텍처가 어떻게 진화하는가

Phase 3 (검토) 직접 Custody

기술:

- KyoboVASPAdapter (교보 자체 구현)
- EVMAAdapter (write 활성화)

키 관리:

- 자체 HSM/MPC
- VASP 인가 취득

컨트랙트:

- 교보 직접 배포·운영

비용 구조:

- VASP 수수료 없음
- 인프라 비용 증가

목표:

- 수수료 절감
- 완전한 데이터 자주권

코드 변화:

- IssuerService: 변경 없음
- VASPAdapter만 교체

이 아키텍처에서 배울 설계 원칙 3가지

원칙 1. 변화 지점을 인터페이스로

변화할 것이 확실한 것:

- 어떤 VASP를 쓸지
- 어떤 체인을 쓸지

→ 변화 지점에 인터페이스
IVASPAdapter
IBlockchainAdapter

→ 비즈니스 로직은
인터페이스만 알면 됨

→ 변화가 생겨도 구현체만 교체

GoF Strategy Pattern 의 실제
적용

원칙 2. 신뢰 경계에서 검증

신뢰할 수 없는 입력:

- 사용자 API 요청
- Webhook 데이터
- 외부 이벤트

→ 경계에서 즉시 검증

- JWT 유효성
- HMAC 서명 확인
- 입력 타입·범위 검증

내부에서는 재검증 없음:
이미 검증된 데이터
→ 재검증 = 낭비

경계를 지나면 신뢰 경계 밖은
의심

원칙 3. 장애를 설계에 포함

장애는 예외가 아니라
정상 동작의 일부

- VASP 장애 → DLQ + 재처리
- Redis 장애 → HA + missed event
- REORG → 롤백 로직
- PENDING stuck → gas bump

'정상 경로'만 설계하고
장애를 나중에 추가하면:
→ 설계 전체 수정

처음부터 장애 경로를 함께
설계해야 한다

이 3가지 원칙은 특정 기술이 아니다. 어떤 분산 시스템에서도 통한다.

Session 2 정리 — 5-Layer 핵심 요약

항목	핵심 내용
레이어 1 당사 직접 운영	EventConditionService·BulkIssueService·mint_requests 상태머신·감사 로그·Gnosis Safe 보조키
레이어 2 DMZ 게이트웨이	202 즉시응답·Redis Streams Consumer Group PEL ACK·DLQ·IVASPAdapter·IBlockchainAdapter·Private RPC
레이어 3 인가 VASP	특금법 VASP 인가·월렛원 외부 위탁·핫/콜드월렛 80-20·MPC 분산 서명
레이어 4 퍼블릭 블록체인	KyoboNFT(ERC1155+AccessControl+Pausable+UUPS)·TransferSingle 이벤트·ADR 001/002
핵심 원칙 1	변화 지점 = 인터페이스. IVASPAdapter + IBlockchainAdapter = 독립적 교체 가능
핵심 원칙 2	신뢰 경계에서만 검증. 내부는 신뢰. 경계 밖은 의심.
핵심 원칙 3	장애를 설계에 포함. DLQ·재처리·REORG 롤백·gas bump = 정상 동작의 일부
다음 세션 S3	스켈레톤 코드 구조 + 의존 방향. npm workspace 모노레포. 패키지 레이어 매핑.